

EECS 182 HW3 – Coding Question (Maximal Update Parameterization)

Claude 4.5 Opus Interaction Report

John Wang

December 7, 2025

Executive Summary

This report documents how I used **Claude 4.5 Opus** to solve the coding portion of HW3, which consists of a single Jupyter notebook implementing and visualizing *Maximal Update Parameterization* (P). I worked in the Claude Code environment where Opus could read and edit the notebook file and maintain a running log of its reasoning, code proposals, and environment-specific fixes in `log.txt`.:contentReference[oaicite:1]index=1

The session covered:

- implementing and interpreting norms of weight matrices and activation updates (Parts A–B),
- coding an optimizer with P-style learning-rate scaling and an equivalent network-side scaling scheme (Parts C–D),
- setting up a hyperparameter transfer experiment (Part E),
- and building a simplified Shampoo/orthogonalization step (Part F).

In addition, because I ran the notebook on an Apple Silicon M3 Max machine, Opus had to help debug MPS-specific issues: converting MPS tensors to NumPy, and working around the lack of SVD support on MPS by moving computations to CPU.:contentReference[oaicite:2]index=2

Overall, Opus was effective at filling in non-trivial PyTorch code and explaining P-related math; however, it exhibited internal tension between different scaling rules (fan-in vs. fan-out), and I relied on both the HW3 instructions and the qualitative behavior of the plots to select a consistent implementation.

1 Setup and Protocol

1.1 Model and Environment

- **Model:** Claude 4.5 Opus (Anthropic).
- **Interface:** Claude Code workspace.
- **Notebook:** `q_mup_coding.ipynb`, the P coding notebook for HW3.
- **Log file:** `log.txt`, which Opus updated with its explanations and code snippets as requested.:contentReference[oaicite:3]index=1

- **Hardware:** Apple Silicon M3 Max, using PyTorch with MPS (Metal Performance Shaders) acceleration when available.

1.2 Initial Instructions to Opus

At the beginning of the coding session I issued two explicit requests (paraphrased from `log.txt`):

You are a helpful coding agent that will work me through the steps of the coding questions in `@q_mup_coding.ipynb`. The file is pre-filled with code and some TODO blocks. Your job is to explain and fill in the TODO blocks. Also, please record my input and your thinking and answers (plus the code) in a file `@log.txt`.

Please explain all parts in more detail, and also fill in the actual notebook cells so I can run them.

Opus acknowledged these instructions, provided background on why P matters, and then proceeded part by part, appending its reasoning and code to `log.txt`.:contentReference[oaicite:4]index=4

1.3 Workflow

The workflow for each conceptual block (Parts A–F) was:

1. I asked Opus to explain the relevant part of the notebook and propose code for the TODO blocks.
2. Opus wrote a conceptual explanation and a concrete code snippet, and appended both to `log.txt`.
3. I copied the code into the actual notebook, ran the relevant cells, and checked for:
 - runtime errors (including MPS-specific issues),
 - whether plots and metrics matched the qualitative behavior described in HW3.
4. If there was an error or mismatch, I pasted the traceback or symptom back to Opus and asked for a minimal fix, which was again recorded in the log.

The final notebook runs end-to-end without errors and produces curves with the expected behavior under standard parameterization and P.

2 Per-Part Interaction and Evaluation

Below I summarize each major section of the log (Parts A–F), focusing on: (1) what the notebook required, (2) what Opus proposed, and (3) how I evaluated or adjusted the solution.:contentReference[oaicite:5]index=

2.1 Part A: Heterogeneous MLP and Activation Changes

Task. Analyze an MLP with heterogeneous hidden sizes `[8, 16, 64, 64, 64, 256, 256, 1024]` by running two forward+backward steps and plotting the RMS of the change in activations for each layer.

Opus explanation. Opus first rewrote the architecture and highlighted that the plotted activation deltas correspond to layers L2–L8 (the internal hidden layers). It explained that with standard initialization and Adam-like updates, the Frobenius norm of weight updates scales as $\sqrt{d_{\text{out}}d_{\text{in}}}$, and the RMS change in activations scales roughly as

$$\text{RMS}(\Delta a_\ell) \propto \sqrt{d_{\text{out}}}.$$

Thus wider layers exhibit larger activation deltas under the same optimizer settings, which is exactly the phenomenon P aims to fix.:contentReference[oaicite:6]index=6

My evaluation. The code around this part was mostly provided; Opus’s role was to interpret the plots and connect them to the scaling argument. Its explanation is consistent with the HW3 writeup and with the empirical behavior I observed when I ran the notebook.

2.2 Part B: Frobenius, Spectral, and RMS–RMS Induced Norms

Task. Compute for each weight matrix update:

1. the Frobenius norm $\|W\|_F$,
2. the spectral norm (largest singular value),
3. an RMS–RMS induced norm that measures how RMS input is mapped to RMS output.

Opus explanation. Opus described each norm:

- Frobenius: scales with $\sqrt{d_{\text{out}}d_{\text{in}}}$.
- Spectral: approximates the worst-case amplification, on the order of $\sqrt{d_{\text{out}}} + \sqrt{d_{\text{in}}}$ for random matrices.
- RMS–RMS induced norm:
 - For input RMS = 1 (so $\|x\|_2 \approx \sqrt{d_{\text{in}}}$), random-matrix heuristics suggest $\|Wx\|_2 \approx \|W\|_F$.
 - Output RMS is then $\|Wx\|_2/\sqrt{d_{\text{out}}}$, and the induced effect can be approximated as $\|W\|_F/\sqrt{d_{\text{in}}}$, i.e. scaling mainly with fan-in.

In code, Opus implemented:

```
frob_norm = torch.norm(p, 'fro').item()
spectral_norm = torch.linalg.svdvals(p)[0].item()
fan_in = p.shape[1]
induced_norm = frob_norm / np.sqrt(fan_in)
```

and used this `induced_norm` as the RMS–RMS measure.:contentReference[oaicite:7]index=7

My evaluation. The derivation is heuristic but internally consistent: both the text and the code use $\|W\|_F/\sqrt{d_{\text{in}}}$. When I plotted these norms against layer width, the RMS–RMS induced norm correlated best with the measured RMS of activation changes, which is the conceptual point of the exercise. I did not further modify this part beyond verifying that the curves matched the qualitative explanations in the notebook.

2.3 Part C: Implementing a P Optimizer

Task. Implement a P-style optimizer that rescales the effective learning rate for each layer so that the RMS of activation updates is (as much as possible) invariant to layer width.

Opus reasoning. Opus derived:

$$\text{RMS}(\Delta a_\ell) \approx \text{lr} \cdot \sqrt{d_{\text{out}}} \text{RMS}(a_{\ell-1}),$$

for Adam-like updates with elementwise $O(1)$ normalized steps. It then reasoned:

- To make this independent of d_{out} , one could scale the learning rate as $1/\sqrt{d_{\text{out}}}$.
- It also recalled that the P paper discusses scaling Adam learning rates inversely with fan-in.
- As a practical compromise, it adopted a “fan-in” heuristic:

```
lr = group['lr']
if len(p.shape) == 2:          # weight matrix
    fan_in = p.shape[1]
    lr = group['lr'] / np.sqrt(fan_in)
elif len(p.shape) == 1:       # bias
    lr = group['lr'] * 0.01
p.add_(u, alpha=-lr)
```

:contentReference[oaicite:8]index=8

My evaluation. This is the part where Opus was most obviously torn between multiple scaling rules (fan-out, fan-in, and $1/\text{fan_in}$). I kept the implemented rule `lr = lr_base / sqrt(fan_in)` and validated it empirically:

- Under standard Adam, the optimal learning rate varies with width and the training curves for different widths do not align.
- Under this P optimizer, the per-width curves line up much more closely and a single learning rate works across widths, which matches the qualitative goal of P.

From a participation standpoint, this is a good example of the model being internally inconsistent in theory but still yielding a useful implementation that I had to sanity-check via experiments.

2.4 Part D: Per-Layer Multipliers in the Network

Task. Implement an alternative scheme where P-style scaling is done inside the network by multiplying each layer’s output by a constant, instead of modifying the optimizer.

Opus reasoning and code. Opus argued that for Adam and sign-based optimizers, scaling the layer output by a constant c scales gradients by c , but the normalization in Adam cancels the magnitude change in the update direction, so the weight-space update is similar while the downstream effect on activations is scaled by c . Thus output scaling can mimic learning-rate scaling.

In code, Opus inserted a per-layer multiplier:

```

for layer in self.layers[:-1]:
    x = layer(x)
    fan_in = layer.weight.shape[1]
    x = x / np.sqrt(fan_in)      # muP-style scaling
    x = self.sigmoid(x)
    activations.append(x)

```

:contentReference[oaicite:9]index=9

My evaluation. I treated this as an alternative to the optimizer-based P: in my runs I either used:

1. a standard optimizer plus these per-layer multipliers, or
2. a P optimizer without extra multipliers,

but not both simultaneously. Under this interpretation the activation change plots across widths behaved as expected.

2.5 Part E: Hyperparameter Transfer Experiment

Task. Conceptually, compare standard Adam and P Adam by training 3-layer MLPs of varying widths, sweeping learning rates and observing how the optimal learning rate depends on width.

Opus explanation. Opus summarized the expected behavior:

- For standard Adam, the optimal learning rate decreases as width grows; no single learning rate works across all widths.
- For the P optimizer, a single learning rate can be used for all widths; the loss curves align, enabling hyperparameter transfer from small to large models.

:contentReference[oaicite:10]index=10

My evaluation. The notebook code for this experiment was mostly provided; Opus's contribution was clarifying the interpretation. Running the notebook confirmed these qualitative differences between standard Adam and the P-style optimizer.

2.6 Part F: Shampoo and Orthogonalization

Task. Implement a simplified Shampoo-like update that orthogonalizes the momentum matrix m via SVD and discuss how this relates Frobenius and spectral norms.

Opus reasoning and code. For non-bias parameters, Opus proposed:

```

if len(m.shape) == 1:
    u = m          # biases: just use momentum
else:
    # SVD-based orthogonalization
    m_cpu = m.cpu()
    U, S, Vh = torch.linalg.svd(m_cpu, full_matrices=False)
    u = (U @ Vh).to(p.device)

```

This sets all singular values to 1, producing an orthogonal update matrix. Opus noted that for such matrices the spectral norm is 1 and the Frobenius norm equals $\sqrt{\min(d_{\text{in}}, d_{\text{out}})}$, so the ratio $\|u\|_F / \|u\|_2$ is fixed.

It also suggested a “bonus” scaling step to normalize the induced norm:

$$u \leftarrow \frac{u}{\sqrt{\min(d_{\text{in}}, d_{\text{out}})}}.$$

My evaluation. For the HW3-scale models, performing a full SVD per step on CPU is acceptable and the orthogonalization behaves as described. The code also correctly handles the MPS constraint that SVD is not implemented on GPU by moving tensors to CPU and back.

3 Apple Silicon / MPS-Specific Issues and Fixes

A substantial fraction of the log deals with making the notebook run smoothly on an M3 Max with MPS acceleration.

3.1 Device Selection and Data Loading

Opus replaced the original Keras-based MNIST loading with TorchVision and added MPS-aware device selection:

```
if torch.backends.mps.is_available():
    device = torch.device('mps')
elif torch.cuda.is_available():
    device = torch.device('cuda')
else:
    device = torch.device('cpu')
```

It used `torchvision.datasets.MNIST` to download and prepare data tensors and moved tensors to `device` in the rest of the code. This resolved Keras import issues and enabled GPU acceleration on Apple Silicon.

3.2 MPS to NumPy Conversion

When plotting activation deltas, I initially encountered:

```
TypeError: can't convert mps:0 device type tensor to numpy.
```

Opus correctly diagnosed that MPS tensors must be moved to CPU before conversion and suggested:

```
deltas = np.array([d.cpu().numpy() for d in activation_deltas_rms])
```

This pattern was applied wherever PyTorch tensors needed to be converted to NumPy arrays for plotting.

3.3 SVD on MPS: CPU Fallback

PyTorch currently does not implement `torch.linalg.svd` and `torch.linalg.svdvals` on MPS. Running those calls on MPS produced:

```
NotImplementedError: The operator 'aten::_linalg_svd.U' is not
currently implemented for the MPS device.
```

Opus worked around this by explicitly moving parameters to CPU for SVD:

```
# For spectral norm in Part B
p_cpu = p.cpu()
frob_norm = torch.norm(p_cpu, 'fro').item()
spectral_norm = torch.linalg.svdvals(p_cpu)[0].item()

# For Shampoo in Part F
m_cpu = m.cpu()
U, S, Vh = torch.linalg.svd(m_cpu, full_matrices=False)
u = (U @ Vh).to(p.device)
```

This allowed the notebook to run end-to-end with MPS acceleration where supported, falling back to CPU only for SVD-based operations.

4 Assessment of Claude 4.5 Opus

Strengths

- **Good PyTorch fluency and environment awareness.** Opus handled typical PyTorch patterns (device handling, MNIST loading, tensor shapes) and quickly adapted the code to Apple Silicon by using MPS where possible and CPU fallbacks where needed.
- **Clear mathematical framing.** For the P-related pieces, it consistently started from the underlying math (how activation RMS depends on layer width) before writing code, which helped me understand the implementation.
- **Debugging assistance.** When confronted with runtime errors (NumPy/MPS conversion, SVD NotImplemented), it localized the source and proposed minimal, targeted fixes rather than rewriting large parts of the notebook.

Weaknesses and Failure Modes

- **Inconsistent scaling stories.** For induced norms and P learning-rate scaling, Opus cycled through several plausible formulas (fan-out, fan-in, $1/\text{fan_in}$, $1/\sqrt{\text{fan_in}}$) and settled on a practical heuristic rather than a uniquely justified rule. I had to verify empirically that the chosen scaling produced the expected curve alignment.
- **Overconfidence in heuristics.** The log shows moments where it declared a choice as a “simple heuristic” even though the assignment has precise definitions. This is a reminder that I cannot treat the model as an oracle for specialized methods like P; the human still needs to enforce the exact specification and sanity-check the results.

Reflection

Using Claude 4.5 Opus for the P coding notebook significantly reduced the mechanical burden of writing PyTorch code and handling Apple-Silicon-specific quirks. At the same time, it highlighted a clear division of labor: the model is strong at local tensor reasoning and environment fixes, but weaker at committing to a single, globally consistent scaling theory.

For this homework, the final notebook is correct in the sense that:

- all cells execute without error on my M3 Max machine, and
- the plots and behaviors match the qualitative expectations in the HW3 handout.

However, achieving this required me to read the P explanations, monitor plots, and override or interpret the models proposed scalings when they conflicted. From a Participation A perspective, this interaction is a concrete example of how a modern LLM can be a powerful coding assistant while still needing a human to enforce problem semantics and theoretical consistency.